TaskFlow

Plateforme de Gestion de Projets Collaborative

Rapport de Conception

Diagrammes UML & Architecture du Système

Projet Fil Rouge — Formation Développement Web

Auteur : Debbarh Mohammed

Date : Année académique 2025–2026

Version : 1.0

Contents

1 Contexte et Présentation du Projet **2**
1.1 Vue d'ensemble 2
1.2 Stack technique 2
1.3 Hiérarchie des rôles (RBAC) 2

2 Diagramme des Cas d'Utilisation **3**
2.1 Vue d'ensemble du diagramme 3

3 Diagramme de Classes **4**
3.1 Vue d'ensemble du diagramme 4
3.2 Note de conception 4

1. Contexte et Présentation du Projet

1.1. Vue d'ensemble

TaskFlow est une application web SaaS de gestion de projets collaboratifs, conçue pour répondre aux besoins des équipes de développement adoptant les méthodologies Agiles (Scrum/Kanban). L'application se positionne comme une alternative légère aux outils complexes du marché, en s'inspirant de Jira pour la rigueur des données, de Linear pour le minimalisme, et de Trello pour la simplicité du tableau Kanban.

Le cœur du système repose sur un tableau interactif permettant de visualiser l'avancement des tâches organisées en **Sprints** et en **Cartes**, avec une hiérarchie stricte de rôles (RBAC) et une intégration d'assistance IA pour la rédaction des tickets.

1.2. Stack technique

Couche	Technologies
Front-End	React / Next.js, Tailwind CSS
Back-End	Laravel (API RESTful), Laravel Sanctum
Base de données	PostgreSQL
IA	API externe (génération de descriptions)

1.3. Hiérarchie des rôles (RBAC)

Le système de contrôle d'accès basé sur les rôles est un élément central de l'architecture. Quatre niveaux d'acteurs sont définis, avec une relation d'héritage (un rôle supérieur hérite des permissions du rôle inférieur) :

Rôle	Permissions principales
Utilisateur	Créer/rejoindre un projet, gérer son profil
Membre	+ Modifier ses cartes assignées, écrire des commentaires
Manager	+ Créer/supprimer des cartes, partager un projet
Owner	+ Gérer le projet (màj/suppression), gérer les membres
Admin	Accès global : statistiques, gestion de tous les utilisateurs

2. Diagramme des Cas d'Utilisation

2.1. Vue d'ensemble du diagramme

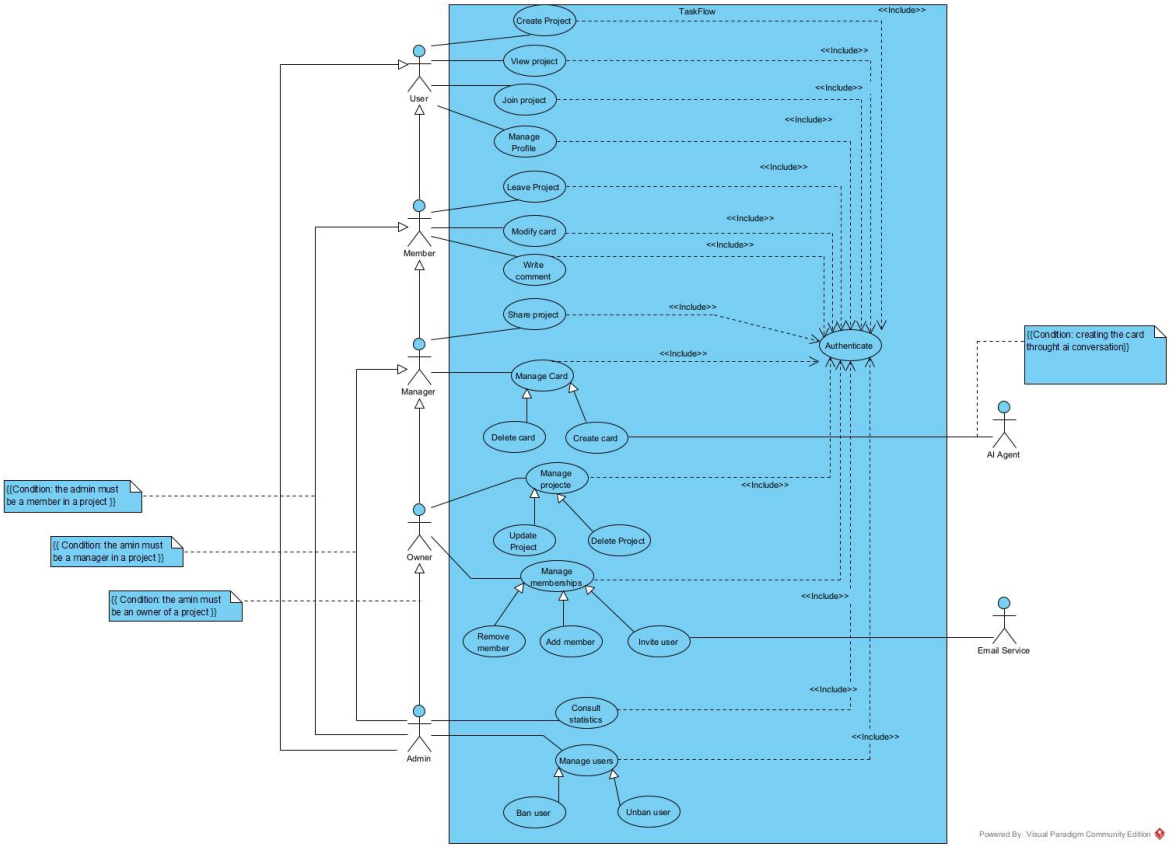

Figure 1: Diagramme des cas d'utilisation — TaskFlow

3. Diagramme de Classes

3.1. Vue d'ensemble du diagramme

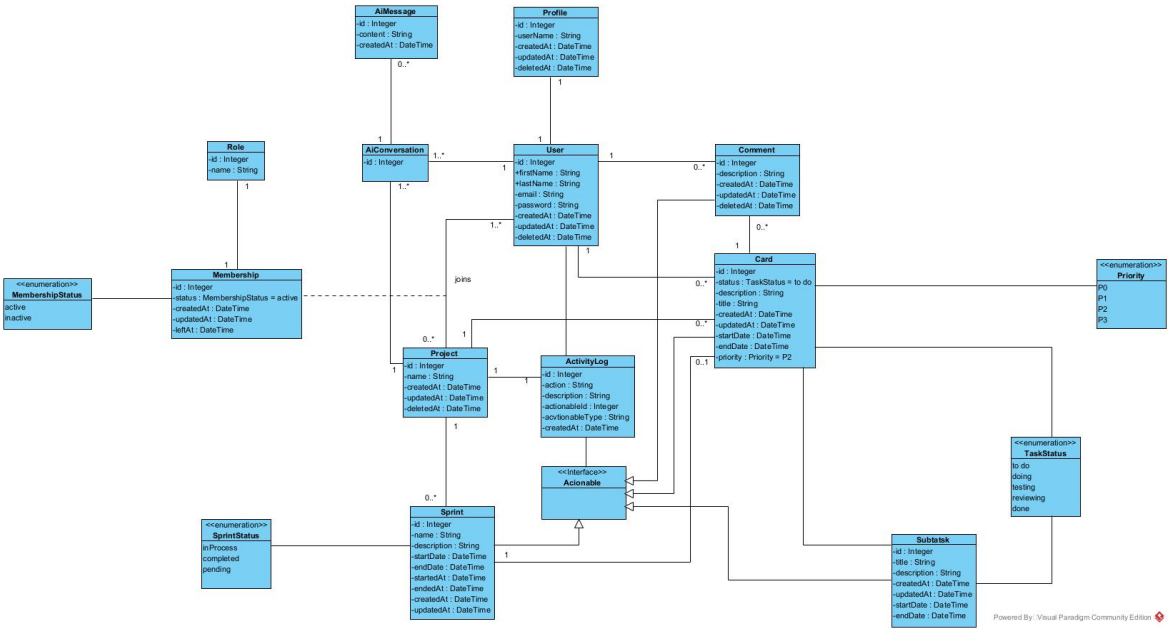

Figure 2: Diagramme de classes — TaskFlow

3.2. Note de conception

`Actionable` : pourquoi une interface pour les logs

`ActivityLog` doit pouvoir enregistrer des actions sur des entités de types différents (`Card`, `Sprint`, `Project`). Plutôt que de créer trois tables de logs distinctes ou trois clés étrangères nullable, l'interface `Actionable` permet de référencer n'importe laquelle de ces entités via deux champs : `actionableId` et `actionableType`. En Laravel, cela se traduit par une relation polymorphe (`morphTo`).